PythonPowerFlow

A Python-based power systems simulation framework for AC power flow analysis, fault studies, and time-series studies.

Second Project for computational power systems at UPitt. Taught by Dr. Robert Kerestes.

Project Overview

PythonPowerFlow is a computational framework designed to simulate and analyze electrical power systems. It provides tools to:

- **Model power networks** with buses, transmission lines, transformers, generators, and loads
- **Compute network admittance** (Y-bus) from component models
- **Solve power flow equations** using Newton-Raphson AC power flow solver to find steady-state voltage and angle profiles
- **Analyze system faults** using Z-bus relationships for symmetric 3-phase faults
- **Simulate time-varying scenarios** by running repeated power flow solves with time-dependent load profiles

The framework is organized into modular classes that represent physical power system components and solvers. All calculations are performed in **per-unit (p.u.)** and **complex** domains where appropriate.

How the Power Flow Simulator Works

What is Power Flow Analysis?

Power flow analysis determines the steady-state operating point of an electrical power system. Given: - Network topology (buses, branches, impedances) - Generation setpoints (active power, voltage control) - Demand (loads)

The solver computes: - Bus voltage magnitudes and phase angles at every bus - Resulting power flows on each branch - System losses and generation requirements

Newton-Raphson Algorithm Overview

PythonPowerFlow uses the **Newton-Raphson iterative method** to solve the nonlinear power flow equations. Here's how it works:

1. **Flat-Start Initialization:**
   - All bus angles $= 0°$
   - All voltage magnitudes $|V| = 1.0$ pu (except fixed setpoints at Slack/PV buses)

2. **Repeated Iterations:**
   - Compute actual power injections at each bus from network equations:

$$P_{calc} = \sum_k |V_i||V_k||Y_{ik}| \cos(\theta_{ik} - \delta_i + \delta_k)$$

$$Q_{calc} = \sum_k |V_i||V_k||Y_{ik}| \sin(\theta_{ik} - \delta_i + \delta_k)$$

   - Calculate power mismatches:

$$\Delta P_i = P_{spec} - P_{calc}$$

$$\Delta Q_i = Q_{spec} - Q_{calc}$$

   - Build the Jacobian matrix (sensitivity of power to voltage changes)
   - Solve the linear system: `J · Δx = f` to find voltage/angle corrections
   - Apply corrections and repeat
3. **Convergence Check:**
   - Stop when max(|mismatch|) < tolerance (default 1e-3)
   - Or max iterations reached (default 50)

Bus Types and Their Roles

The solver handles three bus types with different constraints:

Bus Type	Controls	Solved
Slack	V and	Absorbs system imbalance
PV (Gen)	P and V	Q and computed
PQ (Load)	P and Q	V and computed

Y-Bus: The Network Admittance Matrix

The **Y-bus** is an n×n complex matrix that encodes all series and shunt admittances in the network. Each branch element (line or transformer) contributes a 2×2 primitive admittance matrix that is "stamped" into the appropriate positions of the full Y-bus. See **Y-Bus Matrix Calculation** for details.

Tutorial: Building and Simulating a Circuit

This section demonstrates how to use PythonPowerFlow to model, build, and solve a power system.

Quick workflow (read this first):

1. Create a `Circuit` object.
2. Add buses and assign each bus type (`Slack`, `PQ`, `PV`).

3. Add branches (`TransmissionLine`, `Transformer`) between existing buses.
4. Add injections: generators and loads.
5. Build/update the network admittance matrix with `calc_ybus()`.
6. Run `PowerFlow.solve(...)` with tolerance and max iteration settings.
7. Review convergence, bus voltages, and bus angles from the returned results.

Example 1: Simple 3-Bus System

```
from bus import Bus, BusType
from circuit import Circuit
from powerflow import PowerFlow
from settings import grid_settings
import numpy as np

# Step 1: Create a circuit container
circuit = Circuit("Simple 3-Bus Example")

# Step 2: Add buses (must define nominal voltage and type)
circuit.add_bus("Bus1", nominal_kv=230.0, bus_type=BusType.Slack)
circuit.add_bus("Bus2", nominal_kv=230.0, bus_type=BusType.PQ)
circuit.add_bus("Bus3", nominal_kv=230.0, bus_type=BusType.PV)

# Step 3: Add network branches (transmission lines and transformers)
#         Lines use pi-model with series impedance and shunt susceptance
circuit.add_transmission_line(
    name="Line 1-2",
    bus1_name="Bus1",
    bus2_name="Bus2",
    r=0.01,     # Series resistance (pu)
    x=0.10,     # Series reactance (pu)
    b=0.02      # Shunt susceptance (pu,  -model)
)

circuit.add_transmission_line(
    name="Line 2-3",
    bus1_name="Bus2",
    bus2_name="Bus3",
    r=0.01,
    x=0.10,
    b=0.02
)

# Step 4: Add generators (sources)
circuit.add_generator(
    name="Gen1",
```

```
    bus_name="Bus1",
    mw_setpoint=0.0,              # Slack bus absorbs imbalance
    v_setpoint=1.00               # Slack voltage reference
)

circuit.add_generator(
    name="Gen3",
    bus_name="Bus3",
    mw_setpoint=150.0,
    v_setpoint=1.02
)

# Step 5: Add loads (sinks)
circuit.add_load(
    name="Load2",
    bus_name="Bus2",
    mw=120.0,
    mvar=40.0
)

# Step 6: Build the Y-bus (REQUIRED before solving)
circuit.calc_ybus()

# Step 7: Create solver and solve power flow
solver = PowerFlow()
results = solver.solve(
    circuit,
    tol=1e-4,         # Convergence tolerance
    max_iter=50       # Maximum iterations
)

# Step 8: Inspect results
print("=" * 60)
print("POWER FLOW SOLUTION")
print("=" * 60)
print(f"Converged: {results['converged']}")
print(f"Iterations: {results['iterations']}")
print()

bus_names = results['bus_names']
voltages = results['voltages']
angles_deg = results['angles_deg']

print(f"{'Bus':<10} {'Voltage (pu)':<15} {'Angle (deg)':<15}")
print("-" * 40)
for i, bus_name in enumerate(bus_names):
```

```
    print(f"{bus_name:<10} {voltages[i]:<15.6f} {angles_deg[i]:<15.6f}")
```

Output:

```
============================================================
POWER FLOW SOLUTION
============================================================
Converged: True
Iterations: 4

Bus        Voltage (pu)    Angle (deg)
----------------------------------------
Bus1       1.000000        0.000000
Bus2       0.984315        -2.145892
Bus3       1.020000        1.234567
```

Example 2: 5-Bus System with Time-Series Simulation

Quick workflow for time-series simulation:

1. Build the base circuit (buses, lines, transformers, generators, loads).
2. Build/update the base Y-bus with `calc_ybus()`.
3. Prepare a CSV profile with one step column and one or more load-modifier columns.
4. Create `TimeSeriesPowerFlow()` and load the profile with `load_profile(...)`.
5. Map profile columns to circuit loads with `assign_per_load_modifiers(...)`.
6. Run `run(circuit, tol, max_iter)` to solve power flow at each time step.
7. Review results DataFrame and optionally export with `save_results_csv(...)`.

```
from timeseries import TimeSeriesPowerFlow
from bus import BusType
from circuit import Circuit

# Build the standard 5-bus Example 6.9 network
circuit = Circuit("5-Bus Example")

# Add buses
circuit.add_bus("One",   nominal_kv=15.0,  bus_type=BusType.Slack)
circuit.add_bus("Two",   nominal_kv=345.0, bus_type=BusType.PQ)
circuit.add_bus("Three", nominal_kv=15.0,  bus_type=BusType.PV)
circuit.add_bus("Four",  nominal_kv=345.0, bus_type=BusType.PQ)
circuit.add_bus("Five",  nominal_kv=345.0, bus_type=BusType.PQ)

# Add transmission lines
circuit.add_transmission_line("L42", "Four", "Two", r=0.009, x=0.1, b=1.72)
circuit.add_transmission_line("L52", "Five", "Two", r=0.0045, x=0.05, b=0.88)
circuit.add_transmission_line("L54", "Five", "Four", r=0.00225, x=0.025, b=0.44)
```

```
# Add transformers
circuit.add_transformer("T15", "One", "Five", r=0.0015, x=0.02)
circuit.add_transformer("T34", "Three", "Four", r=0.00075, x=0.01)

# Add generators and loads
circuit.add_generator("G1", "One", voltage_setpoint=1.0, mw_setpoint=278.3)
circuit.add_generator("G3", "Three", voltage_setpoint=1.05, mw_setpoint=520.0)
circuit.add_load("LD2", "Two", mw=800.0, mvar=280.0)
circuit.add_load("LD3", "Three", mw=80.0, mvar=40.0)

# Build Y-bus
circuit.calc_ybus()

# ===== Time-Series Simulation =====
# Create a CSV profile with time-varying load modifiers
import pandas as pd

profile_data = {
    'time': [0, 1, 2, 3, 4],
    'LD2_scale': [1.0, 0.9, 0.8, 0.85, 1.0],  # Load 2 modifier over time
    'LD3_scale': [1.0, 1.0, 1.1, 1.05, 1.0]   # Load 3 modifier over time
}
profile_df = pd.DataFrame(profile_data)
profile_df.to_csv('load_profile.csv', index=False)

# Run time-series power flow
ts_solver = TimeSeriesPowerFlow()
ts_solver.load_profile(
    csv_path='load_profile.csv',
    step_column='time'
)

ts_solver.assign_per_load_modifiers({
    'LD2_scale': 'LD2',
    'LD3_scale': 'LD3'
})

results_df = ts_solver.run(circuit, tol=1e-3, max_iter=50)

# Save results
ts_solver.save_results_csv('timeseries_results.csv')

print(results_df)
```

Example 3: Fault Study

Quick workflow for fault study:

1. Build the circuit and ensure generators include `x_subtransient` values.
2. Build/update Y-bus with `calc_ybus()`.
3. Run a pre-fault power flow to obtain voltage magnitudes (or define a flat 1.0 pu assumption).
4. Build `vprefault_dict` as `{bus_name: voltage_pu}`.
5. Create `FaultStudy()` and call `solve(...)` with:
   - `fault_bus_name`
   - `vf` (fault voltage, complex)
   - `vprefault_dict` (optional but recommended)
6. Review fault current magnitude (`ifn_pu`) and post-fault bus voltages.
7. Inspect `zbus_fault` and `ybus_fault` for deeper analysis.

```
from faultstudy import FaultStudy
from powerflow import PowerFlow

# Use the 5-bus circuit from Example 2
# (assume circuit is already built and solved)

# First, run power flow to get pre-fault voltages
pf = PowerFlow()
pf_result = pf.solve(circuit)

# Extract pre-fault voltage magnitudes
vprefault = {}
for bus_name in circuit.buses:
    vprefault[bus_name] = 1.0  # Simplified; normally use solution from pf_result

# Run fault study
fault_solver = FaultStudy()
fault_result = fault_solver.solve(
    circuit,
    fault_bus_name="Two",
    vf=1.0 + 0.0j,              # Fault voltage (complex)
    vprefault_dict=vprefault
)

print(f"Fault Bus: {fault_result['fault_bus_name']}")
print(f"Fault Current: {fault_result['ifn_pu']:.4f} pu")
print(f"Post-Fault Voltages:")
for bus_name, v_mag in fault_result['post_fault_bus_voltages'].items():
    print(f"  {bus_name}: {v_mag:.6f} pu")
```

Reference Documentation

Bus

Represents a bus (node) in an electrical circuit. Each bus is automatically assigned a unique index upon creation.

Key Attributes:

Attribute	Type	Description
`name`	str	Name identifier (must be non-empty)
`bus_index`	int	Unique auto-assigned index, globally incremented
`nominal_kv`	float	Nominal voltage in kV (read-only)
`v`	float	Current voltage in kV (set by solver)
`bus_type`	BusType	Bus classification: `Slack`, `PQ`, or `PV`
`vpu`	float	Voltage in per unit (default 1.0)
`delta`	float	Phase angle in degrees (default 0.0)

BusType Enumeration:

```
from bus import BusType
BusType.Slack  # Slack bus (voltage and angle fixed)
BusType.PQ     # PQ bus (power specified, voltage/angle solved)
BusType.PV     # PV bus (power and voltage specified, angle solved)
```

Constructor:

`Bus(name: str, nominal_kv: float, bus_type: BusType)`

Methods:

- `_set_voltage(value)`: Sets bus voltage (internal use by solvers only).
- `reset_index_counter()` *(classmethod)*: Resets global index counter to 1 (useful for testing).

Example:

```
from bus import Bus, BusType
Bus.reset_index_counter()
bus1 = Bus("Bus1", 230.0, bus_type=BusType.Slack)
bus2 = Bus("Bus2", 230.0, bus_type=BusType.PQ)

bus1.vpu = 1.00
bus2.vpu = 0.98
```

Load

Load model representing a power consumption element connected to a single bus.

Key Attributes:

Attribute	Type	Description
`name`	str	Load identifier (non-empty)
`bus1_name`	str	Connected bus name (non-empty)
`mw`	float	Real power consumption in MW
`mvar`	float	Reactive power consumption in MVAR
`mva`	float	Apparent power in MVA (computed: $\sqrt{(\text{mw}^2 + \text{mvar}^2)}$)
`p`	float | None	Per-unit real power (set by `calc_p()`)
`q`	float | None	Per-unit reactive power (set by `calc_q()`)

Constructor:

```
Load(name: str, bus1_name: str, mw: float, mvar: float)
```

Methods:

- `calc_p() -> float`: Compute p.u. real power using system base MVA. Updates `p`.
- `calc_q() -> float`: Compute p.u. reactive power using system base MVA. Updates `q`.

Notes:

- `mw` and `mvar` can be any float (positive, negative, or zero).
- `mva` updates automatically when `mw` or `mvar` change.
- `p` and `q` are None until `calc_p()` or `calc_q()` are called.

Example:

```
load = Load("Load1", "Bus2", mw=50.0, mvar=20.0)
print(f"MVA: {load.mva}")  # sqrt(50² + 20²)  53.85
load.calc_p()
load.calc_q()
print(f"p.u. Real Power: {load.p}")
```

Generator

Generator model representing a power source connected to a single bus.

Key Attributes:

Attribute	Type	Description
`name`	str	Generator identifier (non-empty)
`bus_name`	str	Connected bus name (non-empty)

Attribute	Type	Description
`mw_setpoint`	float	Active power setpoint in MW (must be finite)
`v_setpoint`	float | None	Voltage setpoint in p.u. (positive or None)
`x_subtransient`	float | None	Subtransient reactance X''d in p.u. (for fault studies)
`p`	float | None	Per-unit real power (set by `calc_p()`)

Constructor:

```
Generator(name: str, bus_name: str, mw_setpoint: float,
          v_setpoint: float | None = None, x_subtransient: float | None = 1.0)
```

Methods:

- `calc_p() -> float`: Compute p.u. real power using system base MVA. Updates `p`.

Parameters:

- `mw_setpoint`: Must be finite (non-inf, non-nan).
- `v_setpoint`: If provided, must be positive.
- `x_subtransient`: Required for fault studies; typically 0.2–0.3 for synchronous machines.

Example:

```
gen = Generator("Gen1", "Bus1", mw_setpoint=100.0, v_setpoint=1.05, x_subtransient=0.25)
gen.calc_p()
print(f"p.u. Real Power: {gen.p}")
```

Transformer

Represents a transformer modeled as a series impedance (no magnetizing branch). Automatically computes a 2×2 primitive admittance matrix used for stamping into the system Y-bus.

Key Attributes:

Attribute	Type	Description
`name`	str	Transformer identifier (non-empty)
`bus1_name`	str	Side 1 bus name (non-empty)
`bus2_name`	str	Side 2 bus name (non-empty)
`r`	float	Series resistance (pu or ohms)
`x`	float	Series reactance (pu or ohms)
`g`	float	Shunt conductance (default 0)

Attribute	Type	Description
`b`	float	Shunt susceptance (default 0)
`admittance_matrix`	pd.DataFrame	2×2 primitive Y-bus matrix

Constructor:

```
Transformer(name: str, bus1_name: str, bus2_name: str, r: float, x: float,
            g: float = 0, b: float = 0)
```

Primitive Y-bus Structure:

```
          bus1_name    bus2_name
bus1_name   Y_ser       -Y_ser
bus2_name  -Y_ser        Y_ser
```

Where `Y_ser = 1 / (r + jx)`.

Notes:

- Both `r` and `x` cannot be zero simultaneously.
- The admittance matrix updates automatically when impedance properties change.

Example:

```
t = Transformer("T1", "BusA", "BusB", r=0.02, x=0.04, g=0.001, b=0.005)
print(t.admittance_matrix)
t.r = 0.01  # Updates matrix automatically
```

TransmissionLine

Transmission line model with -model representation including shunt susceptance. The total shunt susceptance is split equally at both ends of the line.

Key Attributes:

Attribute	Type	Description
`name`	str	Line identifier (non-empty)
`bus1_name`	str	From-bus name (non-empty)
`bus2_name`	str	To-bus name (non-empty)
`r`	float	Series resistance (ohms or pu)
`x`	float	Series reactance (ohms or pu)
`g`	float	Shunt conductance (default 0)
`b`	float	Total shunt susceptance split b/2 at each end (default 0)
`admittance_matrix`	pd.DataFrame	2×2 primitive Y-bus matrix (-model)

Constructor:

```
TransmissionLine(name: str, bus1_name: str, bus2_name: str, r: float, x: float,
                 b: float = 0, g: float = 0)
```

Primitive Y-bus Structure (-model):

```
          bus1_name          bus2_name
bus1_name  Y_ser + jb/2         -Y_ser
bus2_name   -Y_ser             Y_ser + jb/2
```

Where `Y_ser = 1 / (r + jx)` and shunt is split equally.

Notes:

- Both `r` and `x` cannot be zero simultaneously.
- The -model divides total shunt susceptance `b` as `b/2` at each bus.
- The admittance matrix updates automatically when impedance changes.

Example:

```
line = TransmissionLine("Line1-2", "Bus1", "Bus2", r=0.02, x=0.25, b=0.03)
print(line.admittance_matrix)
line.x = 0.20  # Updates matrix automatically
```

Circuit

Container for a complete power system network. Manages all equipment (buses, branches, generators, loads) and computes the system Y-bus admittance matrix.

Key Attributes:

Attribute	Type	Description
`name`	str	Circuit identifier (non-empty)
`buses`	dict	Bus objects keyed by name
`transformers`	dict	Transformer objects keyed by name
`transmission_lines`	dict	TransmissionLine objects keyed by name
`generators`	dict	Generator objects keyed by name
`loads`	dict	Load objects keyed by name
`y_bus`	pd.DataFrame	System admittance matrix (built by `calc_ybus()`)

Constructor:

```
Circuit(name: str)
```

Primary Methods:

`calc_ybus() -> pd.DataFrame` Constructs the system-wide n×n Y-bus admittance matrix by stamping all branch primitive matrices.

- Returns Y-bus as `pd.DataFrame` with complex values and bus names as index/columns.
- **Must be called after all branches are added** (or after modifications).
- **Required** before power flow solving.

`add_bus(name, nominal_kv, bus_type)` Add a bus to the circuit.

```
circuit.add_bus("Bus1", nominal_kv=230.0, bus_type=BusType.Slack)
```

`add_transformer(name, bus1_name, bus2_name, r, x, g=0, b=0)` Add a transformer between two buses and automatically rebuild Y-bus.

```
circuit.add_transformer("T1", "Bus1", "Bus2", r=0.02, x=0.04)
```

`add_transmission_line(name, bus1_name, bus2_name, r, x, b=0, g=0)` Add a transmission line between two buses and automatically rebuild Y-bus.

```
circuit.add_transmission_line("Line12", "Bus1", "Bus2", r=0.01, x=0.1, b=0.02)
```

`add_generator(name, bus_name, mw_setpoint, v_setpoint=None, x_subtransient=1.0)` Add a generator to a bus.

```
circuit.add_generator("Gen1", "Bus1", mw_setpoint=100.0, v_setpoint=1.05)
```

`add_load(name, bus_name, mw, mvar)` Add a load to a bus.

```
circuit.add_load("Load1", "Bus2", mw=50.0, mvar=20.0)
```

Important Notes:

- Buses must exist before adding any connections to them.
- **`calc_ybus()` must be called after adding all branches** (transformers and lines) before solving.
- Adding branches automatically triggers Y-bus rebuild; explicitly call `calc_ybus()` after major changes.

Example:

```
circuit = Circuit("My 5-Bus System")

# Add buses first
circuit.add_bus("Bus1", 138.0, bus_type=BusType.Slack)
circuit.add_bus("Bus2", 138.0, bus_type=BusType.PQ)
circuit.add_bus("Bus3", 15.0, bus_type=BusType.PV)

# Add branches (Y-bus updated automatically)
circuit.add_transmission_line("L12", "Bus1", "Bus2", r=0.02, x=0.25)
```

```
circuit.add_transformer("T23", "Bus2", "Bus3", r=0.005, x=0.05)

# Add generators and loads
circuit.add_generator("G1", "Bus1", mw_setpoint=200.0, v_setpoint=1.00)
circuit.add_generator("G3", "Bus3", mw_setpoint=100.0, v_setpoint=1.05)
circuit.add_load("L2", "Bus2", mw=150.0, mvar=50.0)

# Build Y-bus (final step)
circuit.calc_ybus()

print(circuit)
print(circuit.y_bus)
```

PowerFlow

Newton-Raphson solver for steady-state AC power flow analysis.

Key Attributes:

Attribute	Type	Description
`converged`	bool	True if solver converged within `max_iter`
`iterations`	int	Number of iterations performed
`mismatch_history`	list	Maximum mismatch norm at each iteration
`J1`, `J2`, `J3`, `J4`	np.ndarray	Jacobian matrix blocks
`J`	np.ndarray	Full Jacobian matrix

Constructor:

`PowerFlow()`

Main Method:

`solve(circuit, ybus=None, tol=1e-3, max_iter=50) -> dict` Solves power flow using Newton-Raphson iteration.

Parameters:

- `circuit`: Circuit object with buses, generators, loads.
- `ybus`: (optional) Pre-computed Y-bus; computed from circuit if not provided.
- `tol`: Convergence tolerance for max mismatch (default 1e-3).
- `max_iter`: Maximum iterations (default 50).

Returns: Dictionary containing:

```
{
    "converged": bool,                  # Convergence status
    "iterations": int,                  # Iterations to converge
```

```
    "mismatch_history": list,             # Mismatch at each iteration
    "bus_names": list,                    # Bus order
    "voltages": np.ndarray,               # Voltage magnitudes (pu)
    "angles_deg": np.ndarray,             # Phase angles (degrees)
    "angles_rad": np.ndarray,             # Phase angles (radians)
}
```

Example:

```
pf = PowerFlow()
result = pf.solve(circuit, tol=1e-4, max_iter=50)

if result['converged']:
    print(f"Converged in {result['iterations']} iterations")
    print(f"Voltages: {result['voltages']}")
    print(f"Angles: {result['angles_deg']}")
else:
    print("Failed to converge")
```

Internal Methods (for diagnostics/testing):

- `_get_power_specs(circuit)`: Extract scheduled P and Q from generators/loads.
- `_calc_power_injections(ybus_np, angles, voltages)`: Compute P and Q injections.
- `_compute_mismatch(...)`: Calculate mismatch vector.
- `_get_voltage_setpoints(buses)`: Extract voltage setpoints from buses.

FaultStudy

Analyzes symmetric 3-phase faults using Z-bus relationships.

Key Method:

`solve(circuit, fault_bus_name, vf=1.0+0j, vprefault_dict=None) -> dict` Simulate a solid 3-phase fault at a specified bus.

Parameters:

- `circuit`: Circuit object with generator X''d subtransient reactances.
- `fault_bus_name`: Name of faulted bus (must exist in circuit).
- `vf`: Fault voltage in p.u. (complex, default 1.0).
- `vprefault_dict`: (optional) Pre-fault voltages {bus_name: voltage_pu}. Defaults to 1.0 pu.

Returns: Dictionary containing:

```
{
    "fault_bus_name": str,
```

```
    "vf": complex,                          # Fault voltage
    "ifn": complex,                         # Fault current (complex)
    "ifn_pu": float,                        # Fault current magnitude (pu)
    "znn": complex,                         # Z-bus diagonal at fault bus
    "post_fault_bus_voltages": dict,        # {bus_name: voltage_pu}
    "zbus_fault": pd.DataFrame,             # Fault-condition Z-bus matrix
    "ybus_fault": pd.DataFrame,             # Fault-condition Y-bus matrix
}
```

Notes:

- Generators must have `x_subtransient` defined (typically 0.2–0.3).
- Pre-fault voltages affect load admittance calculations (constant-impedance model).

Example:

```
fault = FaultStudy()
result = fault.solve(circuit, fault_bus_name="Bus2")

print(f"Fault current: {result['ifn_pu']:.4f} pu")
for bus_name, v_mag in result['post_fault_bus_voltages'].items():
    print(f"  {bus_name}: {v_mag:.6f} pu")
```

TimeSeriesPowerFlow

Runs repeated power flow solves across time steps with time-varying load profiles.

> **Full user guide available.** For a step-by-step walkthrough — including how to build the circuit, prepare the input CSV, map load modifiers, and extract results — see TIMESERIES_README.md.

Key Attributes:

Attribute	Type	Description
`profile`	pd.DataFrame	Loaded time-series data
`step_column`	str	Column name identifying time steps
`column_to_load`	dict	Mapping {profile_column: load_name}
`results`	pd.DataFrame	Results from last `run()` call

Primary Methods:

`load_profile(csv_path, step_column='time') -> pd.DataFrame` Load time-series profile from CSV file.

Parameters:

- `csv_path`: Path to CSV file.
- `step_column`: Column identifying time steps (default 'time').

Example:

```
ts = TimeSeriesPowerFlow()
ts.load_profile('load_profile.csv', step_column='time')
```

`assign_per_load_modifiers(column_to_load) -> None` Map profile columns to specific loads for scaling.

Parameters:

- `column_to_load`: Dict mapping {profile_column: load_name}.

Example:

```
ts.assign_per_load_modifiers({
    'LD2_scale': 'LD2',
    'LD3_scale': 'LD3'
})
```

`run(circuit, ybus=None, tol=1e-3, max_iter=50) -> pd.DataFrame` Execute time-series power flow simulation.

Returns: DataFrame with columns:

```
step                          # Time step identifier
modifier_<load_name>          # Load modifier applied
V_<bus_name>_pu               # Bus voltage at this step
angle_<bus_name>_deg          # Bus angle at this step
converged                     # Convergence status
iterations                    # Iterations to converge
max_mismatch                  # Maximum mismatch
```

Example:

```
results_df = ts.run(circuit, tol=1e-3)
ts.save_results_csv('results.csv')
print(results_df)
```

Settings

System-wide configuration (singleton instance `grid_settings`).

Key Attributes:

Attribute	Type	Description
`freq`	float	System frequency in Hz (default 60)

Attribute	Type	Description
`sbase`	float	System base MVA (default 100)

Usage:

```
from settings import grid_settings

print(f"Frequency: {grid_settings.freq} Hz")
print(f"Base MVA: {grid_settings.sbase}")

grid_settings.sbase = 100.0
```

Y-Bus Matrix Calculation

Overview

The **Y-bus** (bus admittance matrix) is an `n × n` complex matrix (where `n` is the number of buses) that encodes the admittance relationships between all buses in the network. It is the foundation of power flow analysis and directly used in solving the power flow equations (Newton-Raphson, Gauss-Seidel, etc.).

Primitive Admittance Matrix

Each branch element (transmission line or transformer) has a **primitive admittance matrix** — a 2×2 DataFrame indexed by the names of the two buses it connects:

```
          bus_i    bus_j
bus_i     Y_ii     Y_ij
bus_j     Y_ji     Y_jj
```

- **Diagonal entries** (`Y_ii`, `Y_jj`): Sum of series admittance plus any shunt admittance (e.g., `b/2` for a -model transmission line).
- **Off-diagonal entries** (`Y_ij`, `Y_ji`): Negative of the series admittance (`-Y_series`).

For a **transformer** (no shunt):

```
Y_series = 1 / (r + jx)

Y_ii = Y_jj = Y_series
Y_ij = Y_ji = -Y_series
```

For a **transmission line** (model with shunt charging `b`):

```
Y_series = 1 / (r + jx)
```

```
Y_ii = Y_jj = Y_series + j(b/2)
Y_ij = Y_ji = -Y_series
```

The primitive admittance matrix captures the local electrical behavior of a single element and serves as the building block for the full system Y-bus.

Building the System Y-Bus

`build_y_bus()` assembles the full system Y-bus by **stamping** each branch's primitive admittance matrix into the correct positions of the global `n × n` matrix:

1. **Initialization:** An `n × n` complex matrix is initialized to zero. Bus names are mapped to matrix indices in the order they were added to the circuit.
2. **Stamping branches:** For each transmission line and transformer, locate buses `i` and `j` from the bus index map, then add the 2×2 primitive matrix entries:

   ```
   Y[i, i] += Y_primitive[bus_i, bus_i]
   Y[i, j] += Y_primitive[bus_i, bus_j]
   Y[j, i] += Y_primitive[bus_j, bus_i]
   Y[j, j] += Y_primitive[bus_j, bus_j]
   ```

3. **Result:** The final matrix is stored as a `pd.DataFrame` with bus names as both row and column labels, accessible via `circuit.y_bus`.

Important: When to Call `calc_ybus()`

> **`build_y_bus()` must be explicitly called after all circuit elements have been added.**
> Adding any new branch element after this call will invalidate the stored Y-bus (`_y_bus` is set to `None`). Call `calc_ybus()` again before using `y_bus`.

```python
# Correct workflow:
circuit = Circuit("Example")
circuit.add_bus("One", 15.0)
circuit.add_bus("Two", 345.0)
circuit.add_transformer("T1", "One", "Two", r=0.0015, x=0.02)
circuit.add_transmission_line("L1", "Two", "Three", r=0.009, x=0.1, b=1.72)

circuit.calc_ybus()          # <-- Must call this to compute Y-bus
print(circuit.y_bus)           # Safe to access after build
```

5-Bus Example

```python
circuit = Circuit("5-Bus System")
circuit.add_bus("One",   15.0)
```

```
circuit.add_bus("Two",   345.0)
circuit.add_bus("Three", 15.0)
circuit.add_bus("Four",  345.0)
circuit.add_bus("Five",  345.0)

circuit.add_transmission_line("Line 4-2", "Four", "Two",  r=0.009,   x=0.1,   b=1.72)
circuit.add_transmission_line("Line 5-2", "Five", "Two",  r=0.0045,  x=0.05,  b=0.88)
circuit.add_transmission_line("Line 5-4", "Five", "Four", r=0.00225, x=0.025, b=0.44)
circuit.add_transformer("T1-5", "One",   "Five", r=0.0015,  x=0.02)
circuit.add_transformer("T3-4", "Three", "Four", r=0.00075, x=0.01)

circuit.calc_ybus()
print(circuit.y_bus)
```